1.

Algorithm: Let the path have vertices v_1, v_2, …, v_n with weights w_1, w_2, …, w_n

Forward Pass

- Initialize an array A of size n
- $A[0] = 0$
- $A[1] = w_1$
- For $i = 2$ to n: $A[i] = max(A[i-1], A[i-2] + w_i)$

Backward Pass

- Initialize an empty set S
- Let $i = n$
- While $i \geq 1$:
  - If $A[i] == A[i-1]$:
    - Vertex v_i was not included in the optimal solution
    - Decrement i by 1
  - Else:
    - Vertex v_i was included
    - Add v_i to S
    - Decrement i by 2 because we cannot take v_{i-1}

Return S

2.

Algorithm:

- Input: Capacity W, items with weights w_i and values v_i
- Data Structure: Let $K[0 \ldots W]$ be a 1D array where $K[w]$ stores the maximum value achievable with capacity w.
- Initialization: Set all $K[w] = 0$

Pseudocode:

```
For w from 1 to W:
    K[w] = 0
    For i from 1 to n:
        If weights[i] <= w:
        val = K[w - weights[i]] + values[i]
        If val > K[w]:
            K[w] = val
Return K[W]
```

Proof of Claims:

- Correctness: The recurrence relation is $K[w] = max_{i:w_1 \leq w}\{K[w - w_i] + v_i\}$. This explores all possible ways to add the last item i to a subproblem. Since subproblems $K[0 \ldots w-1]$ are solved before $K[w]$, the optimal substructure property holds

- Time Complexity: The outer loop runs W times. The inner loop runs n times. The total operations are $W \cdot n$. Therefore, the complexity is $O(nW)$
- Space Complexity: We use a single 1D array of size $W + 1$, so space is $O(W)$

3.

Pseudocode:

```
Algorithm MergeSort3(A, start, end):
    If (end - start) < 2: Return

    third = (end - start + 1) / 3
    mid1 = start + third
    mid2 = start + 2 * third

    MergeSort3(A, start, mid1)
    MergeSort3(A, mid1, mid2)
    MergeSort3(A, mid2, end + 1)

    Merge3(A, start, mid1, mid2, end)

Algorithm Merge3(A, low, mid1, mid2, end):
    Create temp array T of size (end - low + 1)
    i = low, j = mid1, k = mid2, l = 0

    While (i < mid1 AND j < mid2 AND k <= end):
        if A[i] <= A[j] AND A[i] <= A[k]: T[l++] = A[i++]
        else if A[j] <= A[i] AND A[j] <= A[k]: T[l++] = A[j++]
        else: T[l++] = A[k++]

    … [Standard merge of remaining lists] …

    Copy T back to A[low…end]
```

Proof of Running Time

$T(n) = 3T(n/3) + O(n)$

- $3T(n/3)$: We solve 3 subprograms, each of size n/3
- $O(n)$: The Merge3 step iterates through all n elements once to combine them

Using Master Theorem $T(n) = aT(n/b) + f(n)$:

- $a = 3$
- $b = 3$
- $f(n) = O(n)$

Calculating $log_b a = log_3 3 = 1$. Since $f(n) = \Theta(n^{log_b a}) = \Theta(n^1)$ we are in Case 2 of the Master Theorem. So:

$T(n) = \Theta(n \log n)$

4.

Algorithm: Maximizing $a_j - a_i$ where $i < j$, for any specific a_j, we need to know the min value that appeared before it

- Initialize min_val = a_1
- Initialize max_diff = $-\infty$
- Iterate j from 2 to n:
    - Calc current difference: diff = a_j - min_val
    - Update max_diff = max(max_diff, diff)
    - Update min_val = min(min_val, a[j])
- Return max_diff

Proof of Claims:

- Correctness: At any index j, min_val stores $min(a_1 \ldots a_{j-1})$. The calc a_j - min_val represents the max possible profit if we must sell at index j. By iterating through all valid sell positions j and tracking the global max, we guarantee finding the optimal pair.
- Complexity: the algorithm performs a single pass over the array with constant time operations inside the loop
    - Time Complexity: O(n)
    - Space Complexity: O(1)